2 ЧАСА

8. СОЗДАНИЕ ТРИГГЕРОВ И ПОЛЬЗОВАТЕЛЬСКИХ ФУНКЦИЙ

Цель практической работы №8: изучить принципы создания и использования триггеров и пользовательских функций в SQL для автоматизации обработки данных. Научиться тестировать триггеры путем обновления данных и измерять время выполнения запросов.

Постановка задачи:

1. Изучить синтаксис создания триггеров в SQL.
2. Создать пользовательскую функцию, выполняющую определенные операции с данными (например, вычисления или преобразования).
3. Разработать триггер, который будет автоматически обновлять данные или логировать изменения в таблице при выполнении определенных операций (например, INSERT, UPDATE или DELETE).
4. Протестировать работу триггера:
   - Добавить новую колонку в таблицу.
   - Вставить новое значение в таблицу, чтобы проверить корректность выполнения триггера.
5. Замерить время выполнения запроса SELECT всех строк из первоначальной таблицы найденного датасета. Вывести это время выполнения.

Теоретические аспекты триггеров и пользовательских функций

Пользовательские функции в PostgreSQL представляют собой программные модули, создаваемые с использованием языков процедурного программирования (таких как PL/pgSQL). Они позволяют инкапсулировать сложную бизнес-логику, обеспечивают повторное использование кода и расширяют стандартный функционал SQL. Функции могут возвращать различные типы данных - от скалярных значений до сложных табличных структур, что делает их универсальным инструментом для обработки данных. В контексте проектирования баз данных функции особенно ценны для реализации сложных вычислений, валидации данных и создания абстракций над часто используемыми операциями.

Триггеры являются мощным механизмом автоматизации реакций на события в базе данных. Это специальные процедуры, которые автоматически выполняются при наступлении определенных событий - операций INSERT, UPDATE или DELETE. Триггеры могут срабатывать до или после события, а также вместо него, обеспечивая тонкий контроль над процессами модификации данных. Особую значимость имеет возможность создания условных триггеров, которые активируются только при выполнении определенных условий, что позволяет оптимизировать их производительность. В архитектуре баз данных триггеры служат для поддержания целостности данных, реализации сложных бизнес-правил и ведения журналирования изменений.

Взаимодействие функций и триггеров создает основу для построения интеллектуальных систем управления данными. Триггеры выступают в роли так называемых «слушателей» событий, а пользовательские функции - в качестве «обработчиков» этих событий. Такая архитектура позволяет создавать сложные каскадные операции, автоматизировать процессы денормализации и поддерживать актуальность производных данных. Особенно важно, что эта связка работает на уровне СУБД, обеспечивая атомарность и согласованность операций даже в условиях параллельного доступа.

Выполнение практической работы

```
-- 1) Добавляем колонку для счётчика ноутбуков в таблицу "company"
ALTER TABLE "company"
ADD COLUMN "total_laptops" INT DEFAULT 0;

-- 2) Создаём функцию-триггер, которая будет увеличивать счётчик ноутбуков
CREATE OR REPLACE FUNCTION update_company_laptop_count()
RETURNS TRIGGER AS
$$
BEGIN
  -- Увеличиваем счётчик на 1 у нужной компании,
  -- найденной через связь "laptop" -> "model" -> "company"
  UPDATE "company"
     SET "total_laptops" = "total_laptops" + 1
   WHERE "id" = (
     SELECT "company_id"
```

```
    FROM "model"
    WHERE "model"."id" = NEW."model_id"
  );
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
-- 3) Создаём триггер, который срабатывает после вставки записи в "laptop"
CREATE TRIGGER tr_update_company_laptop_count
AFTER INSERT
ON "laptop"
FOR EACH ROW
EXECUTE PROCEDURE update_company_laptop_count();
```

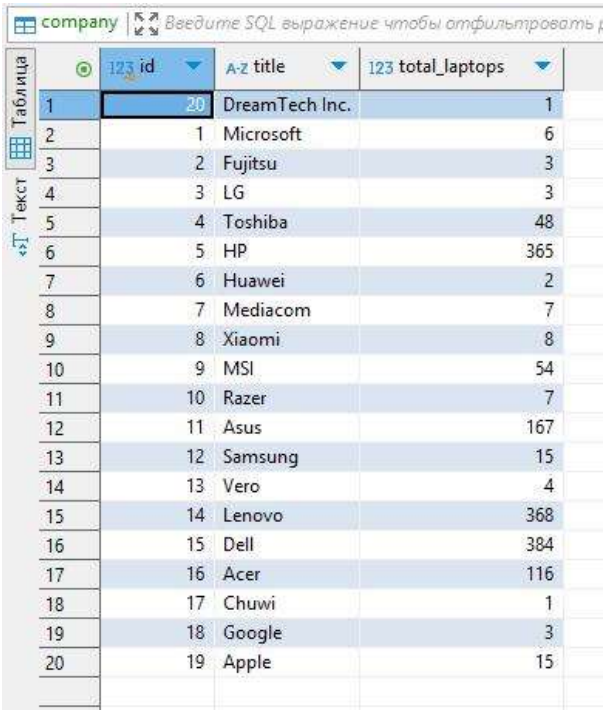
Пояснение по шагам:

ALTER TABLE ... ADD COLUMN – добавляем новую колонку total_laptops в таблицу company, устанавливая её значение по умолчанию равным 0.

CREATE OR REPLACE FUNCTION ... – создаём функцию на языке PL/pgSQL, которая при добавлении новой строки в таблицу laptop будет находить связанную компанию (через таблицу model) и увеличивать счётчик total_laptops на 1.

CREATE TRIGGER ... AFTER INSERT ON ... – определяем триггер, который будет срабатывать после вставки строки в таблицу laptop и вызывать нашу функцию update_company_laptop_count().

На рис. 42 представлена таблица с компаниями и новой созданной колонкой.



company | Введите SQL выражение чтобы отфильтровать

Таблица | Текст

	id	title	total_laptops
1	20	DreamTech Inc.	1
2	1	Microsoft	6
3	2	Fujitsu	3
4	3	LG	3
5	4	Toshiba	48
6	5	HP	365
7	6	Huawei	2
8	7	Mediacom	7
9	8	Xiaomi	8
10	9	MSI	54
11	10	Razer	7
12	11	Asus	167
13	12	Samsung	15
14	13	Vero	4
15	14	Lenovo	368
16	15	Dell	384
17	16	Acer	116
18	17	Chuwi	1
19	18	Google	3
20	19	Apple	15

Рисунок 42. Вывод таблицы компаний

Далее создадим функцию для расчёта количества ноутбуков.

```
-- 1) Создаём функцию для пересчёта количества ноутбуков
CREATE OR REPLACE FUNCTION recalc_company_laptop_count()
RETURNS void
LANGUAGE plpgsql
AS $$
BEGIN
  /*
    Пересчитываем количество ноутбуков для каждой компании
    по связям: company -> model -> laptop
    и записываем в поле total_laptops.
  */
  UPDATE "company"
  SET "total_laptops" = COALESCE((
    SELECT COUNT(l."id")
    FROM "model" m
    JOIN "laptop" l ON l."model_id" = m."id"
    WHERE m."company_id" = "company"."id"
  ), 0);
END;
$$;
```

-- 2) Пример вызова функции (например, после массовой загрузки ноутбуков)
SELECT recalc_company_laptop_count();

Вывод функции recalc_company_laptop_count() представлен на рис. 43.

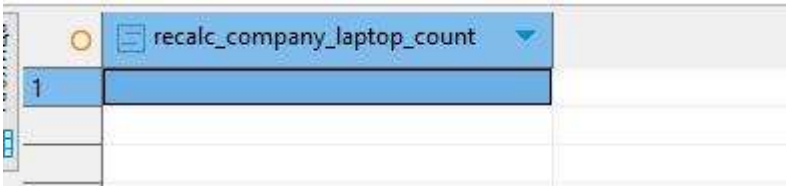


Рисунок 43. Вывод данных по созданию функции

Вывод данных из таблицы «Компания» после запуска функции показан на рис. 44.

company | Введите SQL выражение чтобы отфильтровать

Таблица | Текст

	id	title	total_laptops
1	1	Microsoft	6
2	2	Fujitsu	3
3	3	LG	3
4	4	Toshiba	48
5	5	HP	365
6	6	Huawei	2
7	7	Mediacom	7
8	8	Xiaomi	8
9	9	MSI	54
10	10	Razer	7
11	11	Asus	167
12	12	Samsung	15
13	13	Vero	4
14	14	Lenovo	368
15	15	Dell	384
16	16	Acer	116
17	17	Chuwi	1
18	18	Google	3
19	19	Apple	15
20	20	DreamTech Inc.	1

Рисунок 44. Вывод данных из таблицы «Компания» после запуска функции

Далее написан скрипт создания нового значения в таблицу ноутбуков. Ноутбук добавлен новой компанией (рисунок 4-5).

```
WITH cte_company AS (
  INSERT INTO "company" ("title", "total_laptops")
  VALUES ('DreamTech Inc.', 0)
  RETURNING "id" AS company_id
```

```
),
cte_model_type AS (
    INSERT INTO "model_type" ("title")
    VALUES ('Ультрабук')
    RETURNING "id" AS model_type_id
),
cte_model AS (
    INSERT INTO "model" ("model_type_id", "company_id", "title")
    SELECT
        cte_model_type.model_type_id,
        cte_company.company_id,
        'Dream X-15'
    FROM
        cte_model_type,
        cte_company
    RETURNING "id" AS model_id
),
cte_laptop_os AS (
    INSERT INTO "laptop_os" ("title")
    VALUES ('Windows 11')
    RETURNING "id" AS laptop_os_id
),
cte_laptop_gpu AS (
    INSERT INTO "laptop_gpu" ("title")
    VALUES ('NVIDIA GeForce RTX 3060')
    RETURNING "id" AS laptop_gpu_id
),
cte_laptop_cpu AS (
    INSERT INTO "laptop_cpu" ("title")
    VALUES ('Intel Core i7-11800H')
    RETURNING "id" AS laptop_cpu_id
),
cte_laptop_screen_type AS (
    INSERT INTO "laptop_screen_type" ("title")
    VALUES ('IPS')
    RETURNING "id" AS laptop_screen_type_id
),
```

```
cte_laptop_screen AS (
    INSERT INTO "laptop_screen" ("laptop_screen_type", "resolution", "inch")
    SELECT
        cte_laptop_screen_type.laptop_screen_type_id,
        '1920x1080',
        '15.6'
    FROM
        cte_laptop_screen_type
    RETURNING "id" AS laptop_screen_id
),
cte_laptop_rom_type AS (
    INSERT INTO "laptop_rom_type" ("title")
    VALUES ('SSD')
    RETURNING "id" AS laptop_rom_type_id
),
cte_laptop_rom AS (
      INSERT INTO "laptop_rom" ("lapatop_type_id", "volume")
      SELECT
          cte_laptop_rom_type.laptop_rom_type_id,
          '512GB'
      FROM
          cte_laptop_rom_type
      RETURNING "id" AS laptop_rom_id
  )
  INSERT INTO "laptop" (
      "model_id",
      "laptop_rom_id",
      "laptop_screen_id",
      "laptop_os_id",
      "laptop_gpu",
      "laptop_cpu",
      "price_euro",
      "weight"
  )
  SELECT
      cte_model.model_id,
      cte_laptop_rom.laptop_rom_id,
```

```
    cte_laptop_screen.laptop_screen_id,
    cte_laptop_os.laptop_os_id,
    cte_laptop_gpu.laptop_gpu_id,
    cte_laptop_cpu.laptop_cpu_id,
    '1500',
    '2.2'
FROM
    cte_model,
    cte_laptop_rom,
    cte_laptop_screen,
    cte_laptop_os,
    cte_laptop_gpu,
    cte_laptop_cpu
RETURNING *;
```

Вывод новых значений в таблицы ноутбуков представлен на рис. 45.



	123 id	123 model_id	123 laptop_rom_id	123 laptop_screen_id	123 laptop_os_id	123 laptop_gpu	123 laptop_cpu	A-Z price_euro	A-Z weight
1	1 578	661	40	88	11	111	115	1500	2.2

Рисунок 45. Вывод нового значения в таблицы ноутбуков

Вывод данных компаний ноутбуков представлен на рис. 46.

	123 id	A-Z title	123 total_laptops
1	20	DreamTech Inc.	1
2	1	Microsoft	6
3	2	Fujitsu	3
4	3	LG	3
5	4	Toshiba	48
6	5	HP	365
7	6	Huawei	2
8	7	Mediacom	7
9	8	Xiaomi	8
10	9	MSI	54
11	10	Razer	7
12	11	Asus	167
13	12	Samsung	15
14	13	Vero	4
15	14	Lenovo	368
16	15	Dell	384
17	16	Acer	116
18	17	Chuwi	1
19	18	Google	3
20	19	Apple	15
21	21	DreamTech Inc.	1

Рисунок 46. Вывод данных компаний ноутбука

Время выполнение ≈ 0,097 секунд, можно увидеть на рис. 47.

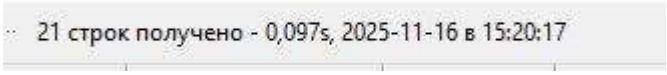


Рисунок 47. Вывод времени обработки запроса

В ходе практической работы №8 были рассмотрены ключевые аспекты создания и использования триггеров и пользовательских функций в PostgreSQL. На практическом примере базы данных ноутбуков разработана функция update_company_laptop_count() для автоматического пересчета количества ноутбуков компаний и создан соответствующий триггер tr_update_company_laptop_count. Проведено тестирование работы триггера путем добавления новой записи в таблицу ноутбуков, что подтвердило корректность автоматического обновления счетчика в связанной таблице компаний. Дополнительно создана функция recalc_company_laptop_count() для массового пересчета статистики, что демонстрирует различные подходы к решению одной задачи.

Для магистрантов института информационных технологий приобретенные навыки имеют фундаментальное значение в контексте дисциплины «Проектирование и разработка баз и хранилищ данных». Умение проектировать и реализовывать триггеры и пользовательские функции позволяет создавать интеллектуальные, саморегулирующиеся системы управления данными, способные автоматически поддерживать целостность и актуальность информации. Эти компетенции особенно востребованы при разработке сложных бизнес-приложений, систем реального времени и хранилищ данных, где требуется обеспечение высокой степени автоматизации и минимального человеческого вмешательства в процессы управления данными.

Практическая часть, посвященная СУБД Postgres Pro охватила полный цикл проектирования и разработки баз данных - от начального анализа предметной области до создания сложных механизмов автоматизации. В рамках работ 1-3 была построена устойчивая архитектура базы данных: проведена нормализация до высших нормальных форм, создана логическая и физическая модели, реализована структура на языке DDL. Работы 4-6 были посвящены освоению SQL как инструмента исследования данных: от базовых операций выборки и модификации до сложных запросов с подзапросами и агрегацией. Завершающие работы 7-8 продемонстрировали возможности

PostgreSQL по созданию интеллектуальных компонентов - представлений, CTE, функций и триггеров.

Полученный опыт формирует комплексное понимание современных подходов к проектированию и разработке систем управления данными. Магистранты не только освоили технические аспекты работы с PostgreSQL, но и развили системное мышление, необходимое для принятия архитектурных решений. Умение обоснованно выбирать уровни нормализации, проектировать эффективные запросы, создавать автоматизированные механизмы поддержания целостности данных - все это составляет основу профессиональной компетенции специалиста в области информационных технологий. Приобретенные знания и навыки служат надежным фундаментом для последующего выполнения практических задач в рамках дисциплины, а также проведения научно-исследовательской деятельности и работы над реальными проектами в индустрии ИТ.

Контрольные вопросы

1. Что такое триггер в SQL, и какие операции он может отслеживать?
2. Как создать пользовательскую функцию в SQL, и для чего она используется?
3. Какой синтаксис используется для создания триггера, реагирующего на операции INSERT, UPDATE или DELETE?
4. Как проверить корректность работы триггера при добавлении новой колонки и вставке данных?
5. Как измерить время выполнения запроса SELECT и вывести его результат?